

Vero Availability Monitor 3: setup and user guide

A small local tool that answers one question all day long: is Vero available right now? It checks Vero on a timer and shows the answer, with its history, on a status page.

Version	3.0.0 (replaces the 0.x latency monitor)
Story	ASPF-1578: <i>As a Quality Lead, I want to monitor latency in the VERO CLI responses.</i> Refocused on availability (ADR-002).
Owner	<< OWNER >> (Quality AI Automation, GenAI Group 3)
Based on	SCMP v0.5 (ASPF-1619) and <i>Vero Chat and Vero CLI: integration guide</i> , draft 0.1 (measured behaviour of Vero CLI 2.3.3 and WChat MCP Server 4.0.3)
Runs on	Windows 10/11 laptop (also Linux and macOS), Python 3.8+, no admin rights, nothing to pip-install

1 Overview

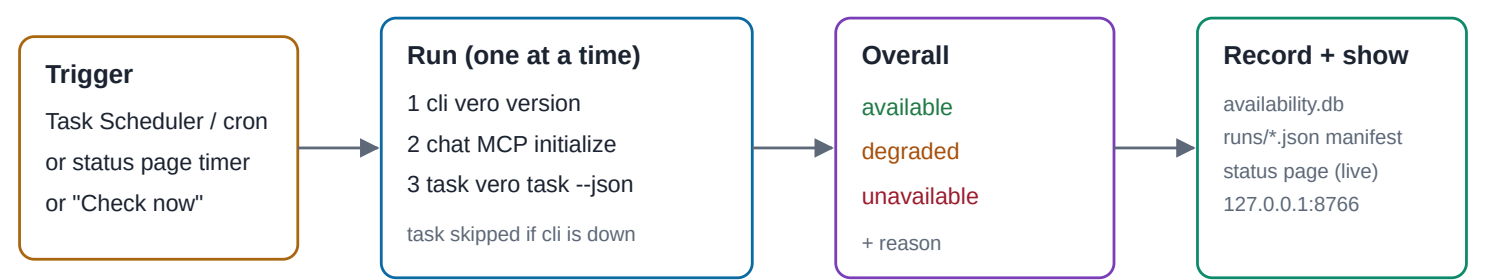
Each run makes up to three checks, cheapest first:

Check	What it does	Proves	Cost
Vero CLI cli	vero version	Vero is installed and starts	free, about 1 s
Vero task task	vero task --json -m <model> -t 180 -c <empty folder> "Reply with exactly: OK"	The whole chain works: CLI, credentials, provider, model and agent loop. This is the check that decides whether Vero counts as available.	2 small model calls, about 1 min
Vero Chat chat (optional)	MCP initialize handshake with verostudio.sw.nxp.com/mcp	The Vero Chat endpoint is up and accepts your token	free, no model call

Acceptance criterion	How V3 meets it
----------------------	-----------------

Should execute the cheapest prompt	One fixed, harmless prompt on a pinned cheap model (Haiku 4.5 by default), plus two checks that cost nothing.
Should support time trigger setup	<code>setup.bat</code> installs a Windows Task Scheduler task that runs on battery and catches up after sleep, or a cron entry on Linux/macOS. The status page can also run the timer itself.
Record and dashboard data	Every run goes into SQLite and gets a JSON run manifest. The live status page shows the current status, availability for 24 h, 7 d and 30 d, a timeline and incidents. You can export CSV or a snapshot.
Should be automated	After setup nothing needs a person: scheduled checks, the status page starts at logon, old data is pruned and logs rotate.

2 How it decides



Every arrow is local: the only network traffic is what Vero itself sends, plus the optional Vero Chat handshake.

Figure 1. One monitoring run.

Check	available	degraded	unavailable
cli	exit code 0	–	not found, exit code ≠ 0, or 30 s timeout
task	a <code>completion_result</code> event arrives	slower than 90 s, or the answer does not contain "OK" (for example a guardrail refusal)	no <code>completion_result</code> , an error exit, or the 210 s hard limit (the process tree is killed)
chat	the MCP <code>initialize</code> result arrives	it takes longer than 10 s	token variable missing, HTTP 401/403, unreachable, an MCP error, or a reply that is not MCP

Overall status of a run:

- `unavailable` if the main check is down. The main check is the task when it is enabled, otherwise the CLI.

- **degraded** if any other check is down, or any check is degraded.
- **available** otherwise.

Availability is the share of runs that were not unavailable. An **incident** is a stretch of consecutive runs that were not fully available. It counts only if it includes an outage, or if the degradation lasts at least 2 runs, so a single slow answer does not flood the list (it still shows on the timeline). An incident ends at the next available run.

The task check follows the measured Vero CLI contract (integration guide §3.3):

- the answer is **only** the `completion_result` event, and `partial` fragments are ignored;
- the real provider and model are taken from `modelInfo` and written to the run manifest;
- `api_req_started`, which contains the full upstream prompt, is never stored or logged.

3 Place in the platform

SCMP / integration guide	In V3
Run manifest: what produced a figure (SCMP §2)	One JSON per run: run_id, tool version, the CLI version, the provider and model reported by Vero next to the configured model, and every check with its status, duration and reason.
Run ID <code><YYYYMMDD-HHMM>-<component></code>	<code>20261001-1930-vero-availability</code>
Common repository skeleton, the four AI documents, ADRs	README, AGENTS, CONTRIBUTING, CODEOWNERS, CHANGELOG, <code>docs/</code> (PRD, ARCHITECTURE, DESIGN_SYSTEM, <code>decisions/ADR-001..004</code>), <code>src/</code> , <code>tests/unit integration fixtures</code> , <code>config/</code> , <code>scripts/</code> , <code>.ci/</code>
Guide §6: "Platform availability monitoring → Vero CLI"	This is that job. It uses the guide's recommended practices: pinned <code>-m</code> , <code>-t</code> always set, <code>modelInfo</code> recorded, no <code>--yolo</code> , a fake CLI so tests never spend tokens.
Guide §9 open decision 1: "Is Vero an approved component?"	Still open. The SCMP does not mention Vero. V3 only <i>observes</i> Vero; it does not make the platform depend on it.
Operational records (SCMP §10)	Running permanently means a row in <code>servers.md</code> . If the <code>chat</code> check uses a functional account token, add it to <code>functional-accounts.md</code> .

4 Requirements

- **Python 3.8 or newer.** Install it from Software Center, from python.org (tick *Add python.exe to PATH*), or with `winget install Python.Python.3.12`.
- **Vero CLI** installed (`npm`), so it resolves to `vero.cmd`) and authenticated (`vero auth`) for the Windows user who will run the monitor.
- Optional, for the Vero Chat check: a Vero Chat token in a user environment variable (section 6).
- No admin rights. The status page listens on `127.0.0.1:8766` only. Data is about 2 MB for 90 days.

5 Install

- 1 **Get the folder** into `C:\Tools\vero-availability-monitor`. Use a local folder, not Downloads or OneDrive. There are two ways:
 - **Git (recommended):** clone it from Bitbucket project QAI << `repo URL` >>.
 - **Source PDF** (for channels that accept only documents): give `VeroAvailabilityMonitor_3.0.0_Source.pdf` to Vero CLI or another AI assistant with: *"Follow the instructions on page 1 of this PDF and rebuild the project in C:\Tools"*. Every source line is numbered, and a visible space mark keeps indentation safe through text extraction. A verify script with every file's SHA-256 confirms the rebuild is byte-identical; tested with two different PDF text extractors. The same content also exists as `VeroAvailabilityMonitor_3.0.0_REBUILD.md`.
- 2 **Optional demo:** double-click `setup_demo.bat`. It uses a fake Vero CLI and seven days of sample data, kept in `data-demo\`.
- 3 **Double-click** `setup.bat`. It asks seven questions; Enter keeps each default. Then it runs `doctor`, one real check that stores nothing (the task part takes about a minute), and asks *Install background monitoring? [Y/N]*.
- 4 **Y** installs the scheduled check and the status page at logon, puts a **Vero Status** shortcut on the desktop, and opens the page. Check everything any time with `status.bat`.

6 Connect Vero

1. Vero CLI program (name or full path; ``where vero`` shows it) [vero]:
found: `C:\Users\me\AppData\Roaming\npm\vero.cmd`
2. Model id to pin (-m), cheapest first [`us.anthropic.claude-haiku-4-5-20251001-v1:0`]:

- 3. End-to-end task check (~1 min, 2 small model calls per run) (y/n) [y]:
- 4. Also check Vero Chat (MCP handshake, free; needs a token variable) (y/n) [n]:
- 5. Check every N minutes [15]:
- 6. Task timeout, seconds [180]:
- 7. Task counts as slow (degraded) above, seconds [90]:

- **Model:** pin the cheapest model your Vero allows. The run manifest still records the model Vero actually used, which a stored task or a configuration change could override.
- **Task arguments:** the default is `task --json -m {model} -t {timeout} -c {workdir} {prompt}`, in `vero.task_args`. `--json` is required. `{workdir}` is the empty folder `data\sandbox`.
- **No `--yo!o`** (ADR-003). The probe only needs the agent to complete, not to touch files or run commands.
- **Timing:** the defaults are based on the measured figures (p50 52.7 s, p95 59.2 s). Interval and timeout are validated together, so a run can never be longer than the interval.

Vero Chat token (only if the chat check is enabled)

The token is never written to the config. The check reads it at run time from the environment variable named in `chat.token_env` (default `VERO_CHAT_TOKEN`). Set it once for your user, then sign out and in again:

```
setx VERO_CHAT_TOKEN "<token from Vero Studio>"
```

A user environment variable is readable by programs running as you. Follow your security policy. For permanent monitoring, use a functional-account token and record it in `functional-accounts.md`, as the SCMP and guide §9 decision 5 require.

7 Time trigger

Task Scheduler task	Runs	Settings
VeroAvailabilityCheck	<code>pythonw.exe vam.py check --trigger task --quiet</code> every N minutes, on :00, :15, :30, :45	Runs on battery and keeps running if unplugged; catches up after sleep; never two at once; 1-hour limit; current user; least privilege
VeroAvailabilityDashboard	<code>pythonw.exe vam.py serve --no-scheduler</code> at logon	Hidden, no time limit, restarts up to 3 times on failure

The tasks are registered from XML with `schtasks /Create /XML`. `install` also sets `run_in_dashboard=false`, so an opened status page never checks twice. Without installing,

`start_dashboard.bat` checks on the same grid while its window is open. On Linux and macOS, `install` adds two crontab lines tagged `# vero-availability-monitor` instead.

8 Status page

Vero status

✓

Vero is available

since Oct 01, 07:20 PM (10 min) · last check 5 s ago

● Vero CLI

available

installed and starts (vero version)
vero 2.3.3 (fake0000) · took 0.1 s · checked 5 s ago

● Vero task

available

end-to-end answer through the model
model us.anthropic.claude-haiku-4-5-20251001-v1:0 · took 3.1 s · checked 5 s ago

● Vero Chat

off

MCP handshake (no model call)
not enabled

Availability, last 24 h

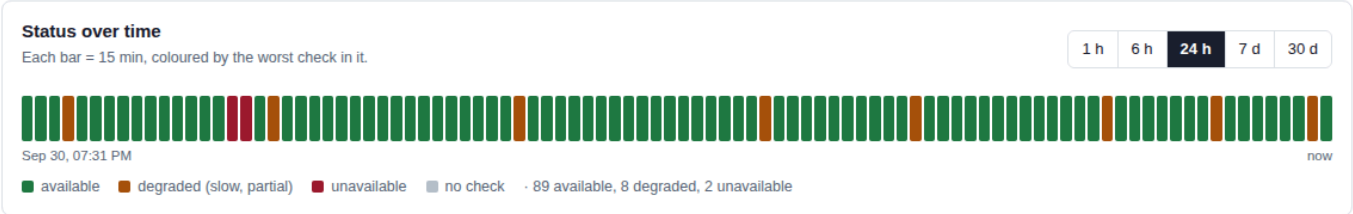
98.0%

Last 7 days

98.5%

Last 30 days

98.5%



Incidents

Periods where Vero was not fully available, newest first.

Started	Duration	Worst	Checks	Reason
Sep 30, 11:20 PM	30 min	unavailable	2	task: no completion within 210 s

Recent checks · live

Time	Result	Checks	Task time	Reason
Oct 01, 07:30 PM	available	● cli ● task	3.1 s	
Oct 01, 07:30 PM	available	● cli ● task	2.4 s	
Oct 01, 07:28 PM	available	● cli ● task	2.8 s	
Oct 01, 07:20 PM	available	● cli ● task	1.6 s	
Oct 01, 07:05 PM	degraded	● cli ● task	108 s	task: slow: 108 s > 90 s
Oct 01, 06:50 PM	available	● cli ● task	60 s	
Oct 01, 06:35 PM	available	● cli ● task	50 s	
Oct 01, 06:20 PM	available	● cli ● task	58 s	
Oct 01, 06:05 PM	available	● cli ● task	59 s	
Oct 01, 05:50 PM	available	● cli ● task	58 s	
Oct 01, 05:35 PM	available	● cli ● task	58 s	
Oct 01, 05:20 PM	degraded	● cli ● task	137 s	task: slow: 137 s > 90 s
Oct 01, 05:05 PM	available	● cli ● task	43 s	
Oct 01, 04:50 PM	available	● cli ● task	46 s	
Oct 01, 04:35 PM	available	● cli ● task	60 s	
Oct 01, 04:20 PM	available	● cli ● task	53 s	
Oct 01, 04:05 PM	available	● cli ● task	47 s	
Oct 01, 03:50 PM	available	● cli ● task	39 s	
Oct 01, 03:35 PM	available	● cli ● task	47 s	
Oct 01, 03:20 PM	degraded	● cli ● task	98 s	task: slow: 98 s > 90 s

Figure 2. The status page with demo data.

Area	Shows
Banner	Vero is available / degraded / unavailable , since when, the time of the last check, and the reason when it is not available. The browser tab title gets ✓ or ⚠.
Component cards	Vero CLI (version), Vero task (model actually used, duration), Vero Chat (server version, or "off")
Availability	Last 24 h, 7 d and 30 d. Green from 99 %, amber from 95 %, red below.
Status over time	Range 1 h, 6 h, 24 h, 7 d or 30 d. Each bar shows the worst result in its time slot; grey means no check ran. Hover a bar for the time and the number of checks.
Incidents	Start, duration (or "ongoing"), worst status, number of checks, reason
Recent checks	Live list with a dot per check, task time and reason. Check now starts a run. Export CSV downloads the selected range.

For a snapshot to email: `python vam.py report --hours 168 --out vero_status.html`. It is one self-contained HTML file.

9 Data and records

```
data/
├─ availability.db           SQLite: runs, checks
├─ monitor.log             rotating log, 1 MB × 4
├─ run.lock                only while a run is in progress
├─ sandbox/                empty working folder for vero task
└─ runs/2026-10/20261001-1930-vero-availability.json
```

```
{
  "run_id": "20261001-1930-vero-availability", "component": "vero-availability", "tool_version":
  "trigger": "task", "host": "NXP-LAPTOP", "started_at": "2026-10-01T19:30:00Z",
  "overall": "up", "reason": "",
  "model": {"provider": "bedrock", "id": "us.anthropic.claude-haiku-4-5-20251001-v1:0",
            "configured_id": "us.anthropic.claude-haiku-4-5-20251001-v1:0"},
  "checks": [
    {"check": "cli", "status": "up", "duration_ms": 1180.4, "detail": "", "version": "vero 2.3."
    {"check": "task", "status": "up", "duration_ms": 52731.0, "detail": "", "version": null}
  ]
}
```

Runs older than `retention_days` (default 90) are deleted after every run, together with their checks. The CSV has one row per check: `run_id, started_at, trigger, overall, check, status, duration_ms, detail, version, provider_id, model_id`.

10 Configuration

The settings are in `config\monitor.json`; `monitor.example.json` holds the defaults. Run `doctor` after every change.

Key	Default	Meaning
<code>vero.command</code>	<code>vero</code>	Program name or full path
<code>vero.model</code>	Haiku 4.5 on Bedrock	Passed as <code>-m</code>
<code>vero.prompt</code> / <code>expect</code>	<code>Reply with exactly: OK</code> / <code>OK</code>	Fixed probe prompt and the text the answer must contain
<code>vero.task_timeout_s</code>	180	Passed as <code>-t</code> . The hard kill comes at <code>timeout + min(30, timeout)</code> seconds.
<code>vero.task_args</code> / <code>version_args</code>	see section 6 / <code>["version"]</code>	Argument lists with the placeholders <code>{model}</code> <code>{timeout}</code> <code>{workdir}</code> <code>{prompt}</code>
<code>vero.env</code>	<code>{}</code>	Non-secret extra variables, for example a proxy
<code>checks.cli</code> / <code>task</code> / <code>chat</code>	<code>true</code> / <code>true</code> / <code>false</code>	Which checks run
<code>chat.url</code> / <code>token_env</code> / <code>timeout_s</code> / <code>ca_bundle</code>	verostudio MCP / <code>VERO_CHAT_TOKEN</code> / 30 / ""	Vero Chat check. <code>ca_bundle</code> is an optional CA file for TLS inspection.
<code>thresholds_s.task_slow</code> / <code>chat_slow</code>	90 / 10	Slower than this counts as degraded
<code>schedule.interval_minutes</code>	15	1 to 1440, and long enough for the worst-case run
<code>schedule.run_in_dashboard</code>	<code>true</code>	Whether the status page runs its own timer. Set automatically by install and uninstall.
<code>retention_days</code> , <code>server.port</code> , <code>data_dir</code>	90, 8766, <code>data</code>	<code>data_dir</code> may use environment variables

11 Commands

Command	Does	Batch file
<code>configure</code>	Guided questions	<code>setup.bat</code>
<code>doctor</code>	All checks once, nothing stored, explained. Exit code 1 if unavailable.	<code>setup.bat</code>
<code>check</code>	One stored run. Exit code 2 if unavailable; always 0 for scheduled runs.	<code>check_now.bat</code>
<code>serve [--open]</code>	Status page. If it is already running, it only opens the browser.	<code>start_dashboard.bat</code>
<code>install</code> / <code>uninstall</code>	Tasks and shortcut; data is kept on uninstall	<code>setup.bat</code> / <code>uninstall.bat</code>
<code>status</code> , <code>open</code> , <code>export</code> , <code>report</code> , <code>init --demo</code>	State; open the page; CSV; snapshot; demo config	<code>status.bat</code> , <code>setup_demo.bat</code>

12 Security

- **No auto-approval:** never `--yolo`, an empty working folder, a fixed harmless prompt (ADR-003).
- **No content kept:** the event stream is parsed in memory. The full upstream prompt (`api_req_started`) and the model answer are never stored. A test proves it with a decoy secret.
- **No secrets stored:** the Vero Chat token comes from an environment variable at run time. Error details are cut to 200 characters, and anything that looks like a credential (bearer, token=, password=, aws_secret) is masked.
- **Local only:** the page binds to 127.0.0.1. Requests for other host names are rejected (no DNS rebinding), and so is "Check now" from other websites (Origin check).
- **Known Vero facts** (guide §3.7): Vero itself keeps task history in `%USERPROFILE%\vero\data\tasks`, and MCP bearer tokens in clear text in `vero_mcp_settings.json`. The monitor's tasks add small entries there; clean them up according to your policy.

13 Troubleshooting

Start with `status.bat`, then `data\monitor.log`, then `python vam.py doctor`.

Reason shown	Fix
<code>cli: 'vero' not found on PATH</code>	Give the full path in configure (<code>where vero</code>); scheduled tasks may have a different PATH.
<code>task: exit 1: ... Unauthorized</code>	Run <code>vero auth</code> as the same Windows user the task runs as.
<code>task: no completion within 210 s</code>	Vero hung, or waited for an approval. Run the task command by hand (<code>doctor</code> prints it as <code>task cmd</code>). If a newer Vero asks for approval even to complete, record it under ADR-003.
<code>task: no JSON events: is --json in vero.task_args?</code>	Restore the default <code>task_args</code> .
<code>task: no completion_result</code>	The agent ended without completing (guide §3.4 retry loop). If it repeats, raise it with the Vero owners.
<code>task: slow: 104 s > 90 s</code>	Vero answered, but slowly (degraded). Adjust <code>task_slow</code> only if the agreed threshold changes.
<code>task: answered, but not with 'OK'</code>	A guardrail or the model answered differently. Vero is up (degraded).
<code>chat: environment variable VERO_CHAT_TOKEN is not set</code>	Run <code>setx</code> (section 6), then sign out and in again.
<code>chat: HTTP 401 (token rejected or expired)</code>	Renew the token.
<code>chat: unreachable: CERTIFICATE_VERIFY_FAILED</code>	Set <code>chat.ca_bundle</code> to the company CA file.
Status page "Reconnecting..."	The page process stopped. Use the desktop shortcut after signing in again, or <code>start_dashboard.bat</code> .
"cannot listen on 127.0.0.1:8766"	Change <code>server.port</code> , then run uninstall and install again.

14 Update, migrate, remove

- **From 0.x (Vero Latency Monitor):** run `uninstall.bat` in `C:\Tools\vero-latency-monitor`, otherwise both tools call Vero every 15 minutes. Its data stays there. V3 uses its own port (8766), task names and database, so the two can also run side by side.
- **Update V3:** run `uninstall.bat`, replace the files (keep `config\monitor.json` and `data\`), then run `setup.bat` (Enter, Enter, ... Y).
- **Remove:** run `uninstall.bat`, then delete the folder. Nothing else is written to the system.

15 Structure and quality

Layer	Modules	Rule
Domain	<code>domain.py</code>	Pure rules (status, availability, incidents, timeline). No I/O, fully unit-tested.
Configuration	<code>config.py</code>	Typed, frozen, validated with clear messages
Adapters	<code>checks/process.py</code> , <code>vero_cli.py</code> , <code>vero_chat.py</code> , <code>store.py</code>	Talk to the outside world: processes, HTTP, SQLite
Application	<code>runner.py</code> , <code>report.py</code>	One run; the status payload
Interfaces	<code>cli.py</code> , <code>server.py</code> , <code>scheduler.py</code> , <code>web/dashboard.html</code>	Commands, HTTP and SSE, timers

Quality gates, all passing for 3.0.0:

- **ruff**: lint, including the security (bandit) and bugbear rule sets, plus formatting.
- **mypy strict**: type-checked for Linux, Windows and macOS.
- **64 unit and integration tests** on Python 3.8, 3.10, 3.11 and 3.13, clean with resource warnings treated as errors. The integration tests use a fake Vero CLI (all failure modes: down, auth, hang, no completion, refusal, slow) and a fake MCP server.
- **Real browser check** with Playwright: the page renders and goes live, "Check now" works, there is no horizontal scroll at 390 px, and the console shows no errors.

Defects found and fixed during the 3.0 build:

- subprocess pipes were not closed (a handle leak in the long-running page process);
- `"env": []` passed validation;
- a POSIX-only call sat on a path the type checker could not prove was never taken on Windows;
- the timeout grace period was too long for short timeouts;
- the CLI being down would also have skipped the Chat check;
- single slow answers flooded the incident list;
- a favicon 404.

16 AC traceability (ASPF-1578)

AC	Evidence	Verified by
Cheapest prompt	The <code>vero.prompt</code> and <code>vero.model</code> settings; free <code>cli</code> and <code>chat</code> checks; the <code>model</code> block in the manifest	<code>doctor</code> output; <code>tests/unit/test_parsers.py</code>
Time trigger	<code>install</code> (Task Scheduler XML / cron); the status page timer	<code>status.bat</code> ; <code>tests/unit/test_scheduler.py</code> ; a screenshot of <code>taskschd.msc</code>
Record and dashboard	<code>availability.db</code> , the run manifests, the status page, CSV, snapshot	<code>tests/integration/test_runner_server.py</code> ; a screenshot of the status page
Automated	Scheduled checks, the page at logon, pruning, log rotation, lock with stale recovery	Several days of unattended runs in the 7 d view

Version history

Version	Date	Change	Author
3.0.0	2026-10-01	Rebuilt as an availability monitor on the measured Vero CLI/Chat contracts; layered structure, strict typing, 64 tests; distributed as source PDF + guide PDF	Elvis-Nicolae Arteni
0.2.0	2026-09-30	Latency monitor: review fixes, guided setup, distribution formats	Elvis-Nicolae Arteni
0.1.0	2026-09-30	Latency monitor, first version	Elvis-Nicolae Arteni